



TÉCNICO SUPERIOR EN DESARROLLO DE APLICACIONES WEB
Departamento de Informática



Sistema de Control de Calidad de Tomate

Documento de Despliegue y Hosting

Autor: Claudio Terrados Sánchez

Curso Académico: 2025/2026

1. Introducción

El presente documento detalla el proceso de despliegue de la aplicación AgroControl en un entorno de hosting en la nube, con el objetivo de que el tribunal pueda acceder y probar la aplicación de forma remota sin necesidad de instalar ningún software en su equipo local.

El despliegue se ha realizado utilizando dos plataformas gratuitas: Railway para el backend (API REST) y la base de datos MySQL, y Vercel para el frontend (interfaz React).

2. URLs de acceso a la aplicación desplegada

La aplicación AgroControl se encuentra accesible de forma pública en las siguientes direcciones:

Componente	URL pública
Aplicación web (Frontend)	https://agro-control-lime.vercel.app
API REST (Backend)	https://agrocontrol-production-895e.up.railway.app

Componente	URL pública
Base de datos (MySQL)	MySQL en Railway (acceso interno desde el backend)

Para acceder a la aplicación, el tribunal solo necesita abrir la URL del Frontend en cualquier navegador web moderno e iniciar sesión con las credenciales indicadas en la sección 3.

3. Credenciales de acceso

Los siguientes usuarios están disponibles en la base de datos desplegada para que el tribunal pueda probar la aplicación:

Rol	Email	Contraseña
Administrador	admin@agrocontrol.local	Admin1234!
Operario	operario@agrocontrol.local	Operario1234!
Operario	maria.gomez@agrocontrol.local	Operario1234!

4. Plataformas utilizadas

4.1. Railway (Backend + Base de datos)

Railway es una plataforma de hosting en la nube que permite desplegar aplicaciones backend y bases de datos de forma sencilla. Se ha utilizado para alojar dos servicios:

Servicio MySQL: Base de datos con las 6 tablas del esquema de AgroControl (usuario, proveedor, variedad, proveedor_variedad, camion y control_calidad), incluyendo los datos de prueba precargados.

Servicio Node.js: API REST desarrollada con Express que se conecta a la base de datos MySQL mediante la red interna de Railway. Railway detecta automáticamente que se trata de un proyecto Node.js y ejecuta el comando npm start para arrancar el servidor.

4.2. Vercel (Frontend)

Vercel es una plataforma de hosting especializada en aplicaciones frontend y frameworks modernos como React, Next.js y Vite. Se ha utilizado para desplegar la interfaz de usuario de AgroControl, desarrollada con React 18 y Vite 5.

Vercel se conecta directamente al repositorio de GitHub y despliega automáticamente cada vez que se realiza un push a la rama principal.

5. Cambios realizados para el despliegue

Para adaptar la aplicación del entorno local al entorno de producción en la nube, se han realizado los siguientes cambios:

5.1. Base de datos

El script SQL original (01a_schema_sin_trigger.sql) creaba una base de datos llamada "agrocontrol". En Railway, la base de datos proporcionada se llama "railway", por lo que se eliminaron las instrucciones DROP DATABASE, CREATE DATABASE y USE agrocontrol del script. Las tablas se crearon directamente dentro de la base de datos "railway" ejecutando el script modificado desde HeidiSQL conectado al servidor MySQL de Railway.

Posteriormente se ejecutó el script de datos de prueba (02_seed.sql) para cargar los registros iniciales, y el script generar-hashes.js para generar los hashes bcrypt de las contraseñas de los usuarios.

5.2. Backend

El backend no requirió cambios en el código fuente. La configuración de conexión a la base de datos ya utilizaba variables de entorno (process.env.DB_HOST, process.env.DB_PORT, etc.), por lo que bastó con configurar las variables de entorno directamente en el panel de Railway:

Variable	Valor	Descripción
DB_HOST	altaria.proxy.rlwy.net	Host interno de MySQL en Railway
DB_PORT	55629	Puerto interno de MySQL
DB_USER	root	Usuario de la base de datos
DB_NAME	railway	Nombre de la base de datos en Railway
JWT_SECRET	agrocontrol_secreto_muy_largo_y_aleatorio_2026	Clave para firmar tokens JWT
NODE_ENV	production	Entorno de ejecución
PORT	3000	Puerto del servidor Express

Adicionalmente, se configuró en Railway el Root Directory como "/backend" para que la plataforma detecte correctamente el package.json del backend, y el Start Command como "npm start".

El servidor Express ya tenía habilitado CORS de forma abierta (app.use(cors())), lo que permite que el frontend desplegado en Vercel pueda comunicarse con la API sin restricciones.

5.3. Frontend

Se creó un archivo de variables de entorno de producción (frontend/.env.production) con la URL pública del backend desplegado en Railway:

VITE_API_URL=https://agrocontrol-production-895e.up.railway.app/api

Esta variable sustituye a la URL local (http://localhost:3000/api) que se utiliza durante el desarrollo. Vite lee automáticamente el archivo .env.production cuando se ejecuta el build de producción.

En Vercel, se configuró el Root Directory como "frontend", el Framework Preset como "Vite", y se añadió la variable de entorno VITE_API_URL con el mismo valor.

6. Proceso de despliegue paso a paso

6.1. Despliegue de la base de datos

1. Se creó una cuenta en Railway vinculada a la cuenta de GitHub del alumno.
2. Se creó un servicio MySQL dentro de Railway, que proporcionó las credenciales de acceso público (host, puerto, usuario y contraseña).
3. Se conectó HeidiSQL al servidor MySQL de Railway utilizando las credenciales públicas.
4. Se seleccionó la base de datos "railway" y se ejecutaron los scripts SQL modificados (sin las instrucciones de creación de base de datos).
5. Se verificó que las 6 tablas se crearon correctamente y se cargaron los datos de prueba.
6. Se ejecutó el script generar-hashes.js apuntando a la base de datos de Railway para generar los hashes bcrypt de las contraseñas.

6.2. Despliegue del backend

1. Desde el dashboard de Railway, se creó un nuevo servicio conectado al repositorio de GitHub (Claud7io/AgroControl).
2. Se configuró el Root Directory como "/backend" en los ajustes del servicio.
3. Se configuró el Start Command como "npm start".
4. Se añadieron las variables de entorno necesarias (DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME, JWT_SECRET, NODE_ENV, PORT).
5. Se generó un dominio público en la sección Networking del servicio, obteniendo la URL: agrocontrol-production-a12c.up.railway.app.
6. Se verificó el correcto funcionamiento accediendo a la URL raíz, que devuelve un JSON con el estado de la API.

6.3. Despliegue del frontend

1. Se creó una cuenta en Vercel vinculada a la cuenta de GitHub del alumno.
2. Se importó el repositorio AgroControl desde GitHub.
3. Se configuró el Root Directory como "frontend" y el Framework Preset como "Vite".
4. Se añadió la variable de entorno VITE_API_URL con la URL del backend en Railway.
5. Se ejecutó el despliegue automático, obteniendo la URL: agrocontrol-ashen.vercel.app.
6. Se verificó el correcto funcionamiento realizando un login de prueba con las credenciales del administrador.

7. Arquitectura del despliegue

El siguiente esquema resume la arquitectura de la aplicación desplegada:

Capa	Tecnología	Plataforma	URL / Acceso
Frontend	React 18 + Vite 5	Vercel	agrocontrol-ashen.vercel.app
Backend	Node.js + Express	Railway	agrocontrol-production-a12c.up.railway.app
Base de datos	MySQL 8	Railway	Red interna (mysql.railway.internal)

El frontend desplegado en Vercel realiza peticiones HTTP a la API REST desplegada en Railway. El backend se conecta a la base de datos MySQL a través de la red interna de Railway, lo que garantiza una conexión segura y de baja latencia.